



University Library Management System

Software Requirements Specification (SRS)

Version	1.0
Date	April 2026
Status	In Development
Stack	Next.js + Bun/Elysia + Prisma + MySQL

Sherif • Noureen • Ahmed • Sayed

1. Project Overview

1.1 Title

LibraFlow — University Library Management System

1.2 Description

LibraFlow is a web-based university library management system designed to streamline the borrowing and returning process for students while giving librarians full control over inventory and fine management.

The system supports three user roles:

- Students — search books, borrow, return, view history, receive notifications
- Librarians — manage catalog, approve borrows, handle fines
- Admins — manage users, view reports, configure system settings

1.3 Goals

- Replace manual/paper-based library operations with a digital system
- Automate fine calculation and due date reminders
- Provide real-time visibility into book availability
- Ensure clean, maintainable code following SOLID principles and Design Patterns

1.4 Scope

In Scope	Out of Scope
Book catalog management	E-book / digital content delivery
Borrowing & returning	Payment gateway integration
Fine calculation	Mobile native app
Email/system notifications	Multi-branch library support
Admin reports	AI-based book recommendations

2. Team Structure

Role	Responsibility	Member
Product Owner + Scrum Master	Requirements, backlog, sprint planning	Sherif
Frontend Developer	UI/UX, Next.js pages, components	Sherif + Noureen

Backend Developer	API, business logic, database	Ahmed + Sayed
-------------------	-------------------------------	---------------

2.1 Design Patterns Used

- Repository Pattern — abstracts all database operations behind a clean interface
- Factory Pattern — creates different borrow types (Book / Magazine / Thesis) with their own rules
- Strategy Pattern — pluggable fine calculation algorithms (Per-Day / Fixed / Percentage)

3. Functional Requirements

3.1 Authentication & Authorization

ID	Requirement
FR-01	The system shall allow users to register with name, email, student ID, and password
FR-02	The system shall allow users to log in using email and password
FR-03	The system shall issue a JWT token upon successful login
FR-04	The system shall support three roles: Student, Librarian, Admin
FR-05	The system shall restrict access to endpoints based on user role
FR-06	The system shall allow users to log out and invalidate their session

3.2 Book Management

ID	Requirement
FR-07	The system shall allow Librarians to add new books (title, author, ISBN, category, quantity)
FR-08	The system shall allow Librarians to edit book details
FR-09	The system shall allow Librarians to delete a book (only if no active borrows exist)
FR-10	The system shall display real-time available quantity for each book
FR-11	The system shall allow all users to search books by title, author, ISBN, or category
FR-12	The system shall support pagination on the book list

3.3 Borrowing System

ID	Requirement
FR-13	The system shall allow Students to request borrowing a book
FR-14	The system shall allow Librarians to approve or reject borrow requests
FR-15	The system shall automatically set a due date based on borrow type (Book: 14 days, Magazine: 7 days, Thesis: 3 days)
FR-16	The system shall use a Factory Pattern to create different borrow types
FR-17	The system shall decrease book available quantity upon borrow approval
FR-18	The system shall allow Librarians to record a book return
FR-19	The system shall increase book available quantity upon return
FR-20	The system shall allow Students to view their full borrowing history
FR-21	The system shall prevent borrowing if the student has an unpaid fine

3.4 Fine System

ID	Requirement
FR-22	The system shall automatically calculate fines when a book is returned late
FR-23	The system shall support multiple fine strategies via Strategy Pattern: Per-Day (5 EGP/day), Fixed (50 EGP after 7 days), Percentage (5% of book price)
FR-24	The system shall allow Librarians to record fine payments
FR-25	The system shall display outstanding fines to Students on their dashboard

3.5 Notification System

ID	Requirement
FR-26	The system shall send a reminder notification 1 day before the due date
FR-27	The system shall notify a Student when their borrow request is approved or rejected
FR-28	The system shall notify a Student when a fine is added to their account
FR-29	The system shall use an Observer Pattern to trigger notifications on borrow events
FR-30	The system shall store all notifications in-system (bell icon)

3.6 Admin & Reports

ID	Requirement
----	-------------

FR-31	The system shall allow Admin to view all users and change their roles
FR-32	The system shall allow Admin to deactivate/reactivate student accounts
FR-33	The system shall provide a report of the most borrowed books
FR-34	The system shall provide a report of currently overdue borrows
FR-35	The system shall provide a summary of total fines collected

4. Non-Functional Requirements

4.1 Performance

ID	Requirement
NFR-01	API responses shall return within 500ms under normal load
NFR-02	The system shall support at least 100 concurrent users without degradation
NFR-03	Book search results shall return within 300ms

4.2 Security

ID	Requirement
NFR-04	All passwords shall be hashed using bcrypt before storage
NFR-05	All API endpoints shall be protected using JWT authentication
NFR-06	Role-based access control shall be enforced on every protected route
NFR-07	The system shall sanitize all user inputs to prevent SQL injection

4.3 Usability

ID	Requirement
NFR-08	The UI shall be responsive and work on desktop and tablet screens
NFR-09	All forms shall display clear validation error messages
NFR-10	The system shall provide a clear loading indicator during API calls

4.4 Reliability

ID	Requirement
NFR-11	The system shall have 99% uptime during working hours
NFR-12	All database transactions shall be atomic (no partial writes)
NFR-13	The system shall gracefully handle errors and return meaningful messages

4.5 Maintainability

ID	Requirement
NFR-14	Code shall follow SOLID principles throughout the codebase
NFR-15	All functions shall be small, single-purpose, and clearly named
NFR-16	No code duplication — shared logic shall be extracted to utilities/services
NFR-17	The system shall include unit tests with minimum 70% code coverage

4.6 Compatibility

ID	Requirement
NFR-18	The frontend shall support the latest versions of Chrome, Firefox, and Edge
NFR-19	The API shall follow RESTful conventions
NFR-20	API responses shall use JSON format

5. Use Cases

Actors

- Student — primary user, borrows and returns books
- Librarian — manages catalog and borrow operations
- Admin — system configuration and reporting
- System — automated actions (reminders, fine calculation)

UC-01: Register & Login

Actor	Student / Librarian / Admin
-------	-----------------------------

Precondition	User has no active session
Main Flow	1. User enters email + password 2. System validates 3. System issues JWT 4. User redirected to dashboard
Alternative Flow	Invalid credentials — System shows error message
Postcondition	User is authenticated with correct role

UC-02: Search for a Book

Actor	Student, Librarian
Precondition	User is logged in
Main Flow	1. User enters search term 2. System queries DB 3. Results displayed with availability
Alternative Flow	No results found — System shows 'No books found' message
Postcondition	User sees list of matching books

UC-03: Borrow a Book

Actor	Student
Precondition	Student is logged in, book is available, student has no unpaid fines
Main Flow	1. Student selects book 2. Submits borrow request 3. Librarian approves 4. BorrowFactory creates record 5. Due date set 6. Quantity decreased 7. Student notified
Alternative Flow A	Student has unpaid fine — System blocks request
Alternative Flow B	Book unavailable — System shows 'Not available'
Postcondition	Borrow record created, student notified

UC-04: Return a Book

Actor	Librarian
Precondition	Active borrow record exists
Main Flow	1. Librarian searches student/book 2. Confirms return 3. System records return date 4. Checks for lateness 5. If late: FineStrategy calculates fine 6. Quantity restored 7. Student notified
Alternative Flow	Returned on time — No fine generated

Postcondition	Borrow closed, fine added if late, stock updated
----------------------	--

UC-05: Pay a Fine

Actor	Librarian
Precondition	Student has an outstanding fine
Main Flow	1. Librarian opens student profile 2. Views outstanding fines 3. Records payment 4. Fine marked as paid 5. Student account unblocked
Postcondition	Student can borrow again

UC-06: Receive Notification

Actor	Student, System
Precondition	A borrow event occurs (approved, rejected, due soon, overdue)
Main Flow	1. Event triggered 2. System stores notification 3. Student sees notification in bell icon 4. Student marks as read
Postcondition	Student is informed of relevant event

6. Definition of Done

Task Level

- Code is written and works locally
- Code follows clean code principles (meaningful names, small functions)
- Feature branch created and pushed to GitHub
- Pull Request opened and reviewed by at least 1 teammate
- PR merged to develop
- No console errors or warnings

Sprint Level

- All sprint stories are completed
- All PRs are merged to develop
- Sprint review demo is recorded or presented
- Backlog is updated for next sprint
- README is updated